

---

## Steelmanning P3552R3

---

Document Number: D4051R0  
Date: 2026-03-17  
Audience: LEWG, SG1  
Reply-to: Vinnie Falco [vinnie.falco@gmail.com](mailto:vinnie.falco@gmail.com)

---

### Table of Contents

---

Abstract

Revision History

R0: March 2026 (post-Croydon mailing)

1. Disclosure

2. The Steelman

3. The Pragmatic Fix

4. What Works

5. The Compound Result Floor

6. I/O in the Sender Pipeline

7. The Convergence

8. The Residual Costs

9. What `std::execution` Gains

10. The Bridge Tax vs. the Sender Tax

11. The Schedule Risk

11.1 The Use-Case Gap

11.2 The Engineering Gaps

11.3 The Unexplored Path

11.4 The Risk

12. Coexistence

13. The Narrowed Gap

14. A Call for Reciprocity

Acknowledgments

## Abstract

---

Grant `std::execution::task` every proposed fix. The result is the coroutine-native model rebuilt inside a sender shell.

This paper steelmans P3552R3<sup>[1]</sup>. It grants every engineering fix that has been proposed or discussed - symmetric transfer, frame allocator timing, error delivery, compound results in the coroutine body, a standardized I/O environment - and traces the strongest possible `std::execution::task` for networking. The steelman works. Type-erased streams, separate compilation, ABI stability, and zero per-operation allocation are all achievable. The I/O hot path is identical to a coroutine-native task. There is no performance gap. The paper then documents what remains: the compound result floor, the vestigial sender machinery, the N-vs-1 bridge tax, and the schedule risk of shipping `task` in C++26 before the fixes land. The gap is small. It is not insurmountable. The paper closes with a call for reciprocity: the author steelmanned `task`. The P2300 community has not steelmanned coroutine-native I/O.

---

## Revision History

---

### R0: March 2026 (post-Croydon mailing)

- Initial version.

---

## 1. Disclosure

---

The author developed and maintains Corosio<sup>[3]</sup> and Cappy<sup>[4]</sup> and believes coroutine-native I/O is the correct foundation for networking in C++. The author provides information, asks nothing, and serves at the pleasure of the chair.

The author regards `std::execution` as an important contribution to C++ and supports its standardization for the domains it serves well - GPU dispatch, heterogeneous execution, and compile-time work-graph composition among them. Nothing in this paper or its companions argues for removing, delaying, or diminishing `std::execution`. The author's position is narrower: that networking and stream I/O present a compound-result

structure that the three-channel model was not designed to carry, and that this domain is better served by a coroutine-native facility that can coexist with senders and interoperate where the domains meet. Two models, each correct for its domain, is a stronger standard than one model asked to serve both.

---

## 2. The Steelman

---

This paper grants P3552R3<sup>[1]</sup> every engineering fix that has been proposed or discussed, assumes they all ship, and asks: what is the best possible `std::execution::task` for networking?

Six fixes are granted:

- a. I/O operations return awaitables, not senders.
- b. Symmetric transfer works task-to-task.
- c. Frame allocator timing is fixed (D3980R0<sup>[5]</sup>).
- d. Error delivery is fixed - no `AS-EXCEPT-PTR` conversion for `error_code`.
- e. Compound results are handled inside the coroutine body via structured bindings.
- f. `IoEnv` is standardized as the networking environment, carrying a type-erased executor and a stop token.

Each fix addresses a documented gap. (a) avoids the template operation state that P4088R0<sup>[6]</sup> Section 6.1 identified as the design fork. (b) resolves the stack overflow vulnerability that P3801R0<sup>[7]</sup> documented. (c) addresses the allocator rework in progress. (d) prevents routine I/O errors from becoming exceptions. (e) keeps compound results out of the three-channel model. (f) provides the executor delivery mechanism that networking requires.

The steelman assumes all six ship. The question is what remains.

---

## 3. The Pragmatic Fix

---

The I/O operation returns an awaitable. The awaitable's `await_suspend` receives a typed coroutine handle:

```

template<class Promise>
auto await_suspend(
    std::coroutine_handle<Promise> h)
{
    io_env_ = h.promise().get_io_env();
    handle_ = h;
    // submit to reactor
}

```

The awaitable extracts the `io_env` from the promise - the type-erased executor and stop token - then stores the handle as `coroutine_handle<>` for resumption. The reactor calls `executor_ref.post(handle_)` when the OS completes. The operation state is concrete. The socket embeds it as a member. Zero per-operation allocation.

For type-erased streams, the vtable needs a fixed function pointer signature. It cannot be templated on the promise type. The promise's `await_transform` wraps the awaitable and injects the `io_env` :

```

// Inside task<T, IoEnv>::promise_type
template<class A>
auto await_transform(A&& aw)
{
    return io_wrapper<A>{
        std::forward<A>(aw), &io_env_};
}

```

The wrapper's `await_suspend` takes plain `coroutine_handle<>` and passes the `io_env` internally. The vtable has a fixed signature. The type-erased stream works.

This works. The awaitable does not know or care whether it is inside `std::execution::task<T, IoEnv>` or a coroutine-native `task<T>` . The `coroutine_handle<>` it stores for resumption is the same opaque pointer. The syscall is the same. The resumption is the same. The frame allocation is the same.

The indirection paths differ. In the coroutine-native model, the promise carries `io_env` directly - the type-erased `executor_ref` is constructed once at task start and passed by pointer to every I/O operation. In the sender model, the promise reaches the scheduler through virtual dispatch on `state_base` (the awaiter inherits from `state_base` and implements `do_get_scheduler()` ), then constructs `executor_ref` on each I/O operation or caches it after the first. The per-operation cost is small - a virtual call and a pointer copy - but the indirection path is longer than a direct pointer to a pre-erased `io_env` .

## 4. What Works

---

The steelman achieves every property that P4088R0<sup>[6]</sup> Section 6 documented as the coroutine dividend:

- **Type-erased streams** ( `any_read_stream` ) with zero per-operation allocation. The vtable dispatches through a fixed signature. The awaitable storage is pre-allocated at construction time and reused for every operation.
- **Separate compilation.** A function accepting `any_read_stream&` goes in a `.cpp` file. Consumers include the header and link. The stream behind the type erasure could be a TCP socket, a TLS session, a file, or a test mock. Nothing recompiles.
- **ABI stability.** The vtable layout does not change. Libraries compiled today work with new transports tomorrow.
- **Symmetric transfer.** O(1) stack depth regardless of chain length, task-to-task.
- **Frame allocator propagation.** The allocator reaches `promise_type::operator new` before the frame is allocated.
- **Compound results via structured bindings.** `auto [ec, n] = co_await sock.read_some(buf)` . Both values visible. No channel split. No data loss.
- **No `co_yield with_error`** . Compound results stay in the coroutine body. The mechanism that P3801R0<sup>[7]</sup> identified as blocking symmetric transfer is simply unnecessary.
- **Structured concurrency.** `when_all` and `when_any` with child lifetime guarantees and stop token propagation.
- **No performance gap on the I/O hot path.** The awaitable, the syscall, the resumption, and the frame allocation are identical to the coroutine-native model.

The steelman delivers everything the coroutine-native model delivers for networking. The author states this without reservation.

---

## 5. The Compound Result Floor

---

Inside the coroutine body, compound results work perfectly. The problem appears when the result must leave the coroutine through the sender interface.

`task<T, IoEnv>` is a sender. Its completion signature is `set_value_t(T)`. When a coroutine handles I/O errors internally and returns normally, the sender interface sees `set_value`. When the coroutine throws, the sender interface sees `set_error(exception_ptr)`. The three-channel model provides `set_value`, `set_error`, and `set_stopped`.

An I/O read returns `(error_code, size_t)`. If the result must cross the sender boundary, three routing options exist:

| Routing                                                                     | Preserves           | Breaks                                                                                                                |
|-----------------------------------------------------------------------------|---------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>set_value(ec, n)</code> - both values on the value channel            | Both values         | <code>upon_error</code> unreachable, <code>retry</code> does not fire, <code>when_all</code> does not cancel siblings |
| <code>set_error(ec)</code> on failure, <code>set_value(n)</code> on success | Composition algebra | Byte count destroyed on error - partial reads lost                                                                    |
| <code>set_value(pair{ec, n})</code> - pair on the value channel             | Both values         | Same as row 1 - composition blind to errors                                                                           |

Kohlhoff identified this in 2021:

*“Due to the limitations of the `set_error` channel (which has a single ‘error’ argument) and `set_done` channel (which takes no arguments), partial results must be communicated down the `set_value` channel.” (P2430R0<sup>[8]</sup>)*

Kühl documented five routing options in P2762R2<sup>[9]</sup>. None preserve both values and retain composition. P4090R0<sup>[10]</sup> traced each option to its consequence. The three-channel model was designed for infrastructure errors where there is an error *or* a value. I/O produces compound results where there is an error *and* a value simultaneously. The model cannot express it.

This is a property of the **standard** algorithms, not of all possible sender algorithms. A custom sender `when_all` whose internal receiver inspects the `set_value` arguments through a predicate could cancel siblings on I/O errors while preserving both values. Both models - sender and coroutine - can build predicate-based combinators with equivalent behavior. The structural limit is that the standard sender algorithms (`when_all`, `upon_error`, `retry`) dispatch on the channel, not on the value. Each standard algorithm that should be I/O-aware would need a custom predicate variant. The coroutine model does not have a parallel set of channel-dispatched algorithms that remain blind.

Inside the coroutine body, the steelman handles compound results correctly. The standard sender composition algebra is not designed for compound I/O results. It is designed for infrastructure errors - allocation failure, scheduling failure, shutdown - where there is an error or a value, never both.

---

## 6. I/O in the Sender Pipeline

---

The steelman keeps I/O inside coroutines. What if someone wants I/O directly in a sender pipeline, with no coroutine frame?

```
sock.async_read_some(buf)
| ex::then([](error_code ec, size_t n) {
    // ...
});
```

Four options exist. All are traced to conclusion.

**Option 1:** The socket returns a sender. `connect(sender, receiver)` stamps the receiver into the operation state. The operation state is a template. It cannot live in the socket. It cannot be type-erased without per-operation allocation. Separate compilation and ABI stability are lost. This is the original design fork from P4088R0<sup>[6]</sup> Section 6.1.

**Option 2:** The sender wraps an internal coroutine. The sender's `start()` launches a coroutine that `co_await`s the awaitable, then calls `set_value` / `set_error` on the receiver. The coroutine frame is allocated. The operation state is still a template. Both costs are paid, neither benefit is gained.

**Option 3:** Callback-based I/O with a sender wrapper. The socket registers with the reactor, gets a callback on completion, wraps it in a sender. No coroutine frame. But the reactor needs a fixed-signature callback to call when the OS completes. The receiver type must be type-erased at the reactor boundary - a function pointer and a `void*`. This is functionally identical to `coroutine_handle<>`, except without the compiler managing the state.

**Option 4:** Type-erase the receiver at the reactor boundary. The reactor stores a `void*` and a function pointer. On completion, it calls through the pointer. The concrete operation state casts back and calls `set_value` / `set_error` on the receiver. This works. But the type erasure is the same cost as `coroutine_handle<>`, and the caller gets none of the compiler-managed state, the `co_await` ergonomics, or the structured bindings on the result.

Every path ends at the same place: either the operation state is a template (losing type erasure, separate compilation, and ABI stability) or the receiver is type-erased at the reactor boundary (reinventing `coroutine_handle<>` by hand). The sender pipeline's full-visibility property - the optimizer seeing the entire chain - breaks at the I/O boundary regardless, because the OS does not template on the receiver type.

---

## 7. The Convergence

The steelman `task<T, IoEnv>` with all fixes and a coroutine-native `task<T>` produce identical behavior for networking. The following table compares them:

| Property                                                 | <code>task&lt;T, IoEnv&gt;</code> | Coroutine-native <code>task&lt;T&gt;</code> |
|----------------------------------------------------------|-----------------------------------|---------------------------------------------|
| Promise carries <code>io_env</code>                      | Yes                               | Yes                                         |
| <code>await_transform</code> injects <code>io_env</code> | Yes                               | Yes                                         |
| Type-erased executor                                     | Yes                               | Yes                                         |
| Symmetric transfer                                       | Yes                               | Yes                                         |
| Compound results in body                                 | Yes                               | Yes                                         |
| Type-erased streams                                      | Yes                               | Yes                                         |
| Separate compilation                                     | Yes                               | Yes                                         |
| ABI stability                                            | Yes                               | Yes                                         |
| I/O hot path performance                                 | Identical                         | Identical                                   |
| Is a sender                                              | Yes                               | No                                          |
| <code>Environment</code> template parameter              | Yes                               | No                                          |
| Sender concept machinery in promise                      | Yes                               | No                                          |
| Standard sender algorithms for I/O errors                | Blind (custom variants possible)  | N/A (not needed)                            |

Every row above the dividing line is identical. The rows below the line are the differences. The steelman `task<T, IoEnv>` is the coroutine-native model rebuilt inside `std::execution::task`. The sender shell is vestigial for networking.

## 8. The Residual Costs

After granting every fix, five costs remain:

- **Compile-time cost.** `task<T, IoEnv>` satisfies the `sender` concept. The promise provides completion signatures, supports `connect / start`, and satisfies `sender_to<Receiver>` for valid receivers. The sender concept machinery is instantiated for every coroutine in the chain, even though networking code never uses the sender interface. A coroutine-native `task<T>` includes `<coroutine>` and almost nothing else.
- **Environment** parameter. `task<T>` (default environment) and `task<T, IoEnv>` are different types. A function returning one cannot be assigned to a variable of the other. Users must know which spelling to use. Library interfaces must choose.
- **Domain mismatch.** `task<T, IoEnv>` is a sender. The standard sender algorithms ( `upon_error`, `retry`, `when_all` ) work correctly for their intended domain - infrastructure errors where there is an error or a value, never both. Compound I/O results are outside that design envelope. The presence of the sender interface invites composition with algorithms not designed for compound I/O results. A coroutine-native task that is not a sender does not present this mismatch.
- **Process coupling.** `IoEnv` must be standardized alongside or after `task`. If `task` ships without `IoEnv`, networking cannot use it. The coroutine-native model has no such dependency.
- **`await_transform` contention.** The promise must handle both sender-to-awaitable conversion (for `co_await` ing senders) and I/O awaitable injection (for `co_await` ing I/O operations). Both transforms coexist in the same promise. A coroutine-native task has only the I/O transform.
- **Task-to-task runtime overhead.** When one `task<T, IoEnv>` `co_await` s another, the `awaiter` in the `beman::task` reference implementation constructs a `state_rep`, extracts the scheduler from the parent's environment via virtual dispatch through `state_base`, and stores the parent handle. On completion, the awaiter compares the current scheduler against the parent's scheduler and potentially reschedules through an additional sender `connect / start` cycle. Per `co_await` of a child task, multiplied by N in a chain. In the coroutine-native model, the same path is: store the continuation handle, symmetric transfer. Two pointer stores. The per-operation overhead is small - nanoseconds - but it is not zero, and it scales with chain depth.

None are showstoppers. All are friction. The question is whether the friction justifies a separate task type.

---

## 9. What `std::execution` Gains

---

If `std::execution` does not carry I/O, the model becomes better for its own domain:

- **Uncompromised pipeline visibility.** No I/O boundary breaking the optimizer. The full-visibility property that makes senders valuable for GPU dispatch and compile-time work graphs is preserved end to end.

- Clean three-channel model. Infrastructure errors only. No compound result ambiguity. `set_error` carries an error. `set_value` carries a value. The channels mean what they say.
- `task<T>` with one parameter. No `IoEnv` fragmenting the type. No `Environment` template parameter. The lingua franca holds.
- Smaller specification surface. No I/O concepts, no reactor integration, no type-erased executors inside the task promise.
- The model stays correct for its domain. GPU dispatch, heterogeneous execution, compile-time work graphs - the domains where `std::execution` is deployed at scale (P2470R0<sup>[11]</sup>) - are unaffected.

`std::execution` becomes stronger by not carrying I/O. The domains it serves well are served better.

---

## 10. The Bridge Tax vs. the Sender Tax

---

In the `task<T, IoEnv>` model, every coroutine in a chain is a sender. Consider a five-coroutine networking chain:

```
task<header, IoEnv>
read_header(socket& sock);

task<message, IoEnv>
read_message(socket& sock);

task<response, IoEnv>
handle_request(socket& sock);

task<void, IoEnv>
write_response(socket& sock, response r);

task<void, IoEnv>
session(socket& sock);
```

Five coroutines. Every one is a sender. Every `co_await` between them goes through the sender-awaitable bridge inside `task`'s promise. Every `co_return` satisfies completion signatures. The sender concept machinery is instantiated five times. None of the intermediate coroutines ever need to be senders - they are passing results up a coroutine chain via `co_await`.

In the coroutine-native model with bridges (P4092R0<sup>[12]</sup>, P4093R0<sup>[13]</sup>):

```

copy::task<header> read_header(socket& sock);
copy::task<message> read_message(socket& sock);
copy::task<response> handle_request(socket& sock);
copy::task<> write_response(socket& sock,
                        response r);
copy::task<> session(socket& sock);

```

Five coroutines. Zero sender machinery. Plain `co_await`, plain `co_return`. The bridge cost is paid once, at the edge, where the coroutine chain meets the sender world:

```

auto sndr = copy::as_sender(session(sock));
ex::sync_wait(sndr);

```

One bridge instead of five sender instantiations.

| Model                                          | Sender instantiations | Bridge cost     |
|------------------------------------------------|-----------------------|-----------------|
| <code>task&lt;T, IoEnv&gt;</code> (every task) | N (every coroutine)   | 0 (built in)    |
| Coroutine-native + bridge at edge              | 0                     | 1 (at boundary) |

For a chain of N coroutines, the `task<T, IoEnv>` model instantiates sender machinery N times. The bridge model instantiates it once. This is a compile-time argument - the sender machinery is mostly compile-time cost - but compile time matters. Template-heavy designs are one of the reasons the committee could not ship the Networking TS.

P2300 does not lose anything by having I/O in a separate domain. It gains, because the bridge is cheaper than making every I/O coroutine a sender.

## 11. The Schedule Risk

The author's papers may have created the perception of an attack on `std::execution`. The author understands why the `std::execution` community would close ranks. That is a natural and reasonable response to what looks like a coordinated challenge to years of work. This paper exists to demonstrate the opposite intent. The steelman itself - Sections 2 through 4, granting every fix, tracing every mechanism, concluding that it works - is proof that the author wants `std::execution::task` to succeed in its domain.

## 11.1 The Use-Case Gap

P3552R3<sup>[1]</sup> motivates `task` as follows: “The expectation is that users would use the framework using some coroutine type.” The paper’s examples are: Hello World ( `co_return co_await ex::just(0)` ), spawn into a `counting_scope` , read a value from the environment, pass an allocator, and demonstrate the stack overflow problem. Every example `co_await` s a sender ( `ex::just()` ). None performs real async work - no file I/O, no networking, no database query, no HTTP request.

No published paper by anyone on the P2300 side shows what a user writes inside `std::execution::task` for a production use case. What does a call site look like? What operations does the user perform inside the coroutine? How does the user launch the task? P3552R3<sup>[1]</sup> assumes `task` is needed but does not analyze what it is needed for. The implementation section acknowledges:

*“This implementation hasn’t received much use, yet, as it is fairly new.” (P3552R3<sup>[1]</sup>, Section 7)*

## 11.2 The Engineering Gaps

The steelman in Sections 2 through 4 assumes six engineering fixes. Kühl’s P3552R3<sup>[1]</sup> is the vehicle. The C++26 cycle is closing. The fixes that the steelman requires are not in P3552R3<sup>[1]</sup> today:

- **Symmetric transfer.** P3801R0<sup>[7]</sup> identified the stack overflow. Kühl is exploring trampolines (P3796R1<sup>[14]</sup>). If the trampoline approach does not land, C++26 ships a `task` with recursive stack growth on every `co_await` chain - a runtime cost and a security vulnerability that cannot be fixed later without changing the promise ABI.
- **Frame allocator timing.** D3980R0<sup>[5]</sup> is a rework published six months after P3552R3<sup>[1]</sup>’s adoption at Sofia. Still in progress.
- **IoEnv** . Does not exist in any proposal. The steelman requires it.
- **Error delivery.** AS-EXCEPT-PTR converting routine `error_code` to `exception_ptr` is still in the specification.

## 11.3 The Unexplored Path

P2762R0<sup>[15]</sup> (“Sender/Receiver Interface For Networking”) mentioned `io_task` in one paragraph:

*“It may be useful to have a coroutine task ( `io_task` ) injecting a scheduler into asynchronous networking operations used within a coroutine... The corresponding task class probably needs to be templated on the relevant scheduler type.” (P2762R0<sup>[15]</sup>)*

That is the entirety of the published exploration. “It may be useful.” “Probably needs to be templated.” P2762<sup>[9]</sup> stopped at R2 (October 2023). No revision in over two years. The paper has not been updated to integrate with P3552R3<sup>[11]</sup>. No published paper by anyone defines what `IoEnv` looks like inside `std::execution::task` - what the promise must carry, how `await_transform` delivers the executor, how type-erased streams work under the design.

This paper is, to the author’s knowledge, the first published exploration of that mechanism. The author of this exploration is not the author of `task`. It is the author of the competing model. The steelman was constructed by the opponent.

That is not a criticism of Kühl. He has been occupied with the engineering gaps documented in P3796R1<sup>[14]</sup>. It is an observation about where the committee’s attention is directed. The path to making `task` serve networking has been explored by the person who believes a separate task type is the better answer. If the committee intends to ship `task` for networking, someone on the `task` side needs to do this work.

## 11.4 The Risk

The political pressure to ship `task` in C++26 - to demonstrate that `std::execution` has a coroutine story - is understandable. But if `task` ships without these fixes, the standard locks in a task type that cannot achieve the steelman. The networking community would then need a second task type anyway - not because of a design preference, but because the shipped `task` is structurally unable to serve networking.

Deferring `task` to C++29 - alongside networking - gives Kühl the time to get it right. Rushing it into C++26 under political pressure risks shipping a task type that the committee must work around for the next decade. The steelman is achievable. The schedule is not.

---

## 12. Coexistence

Bridges between the two models exist and work. P4092R0<sup>[12]</sup> demonstrates consuming senders from coroutine-native code. P4093R0<sup>[13]</sup> demonstrates producing senders from coroutine-native code.

Sender to coroutine:

```

capy::task<int> use_sender()
{
    co_return co_await as_awaitable(
        some_sender);
}

```

Coroutine to sender:

```

auto sndr = capy::as_sender(
    some_io_task())
| ex::then([](auto result) { /*...*/ });

```

Two models. Neither compromises the other. The bridge cost is paid once at the boundary, not N times through the chain (Section 10).

## 13. The Narrowed Gap

The steelman granted every fix. It traced the mechanism. It showed that `task<T, IoEnv>` can achieve type-erased streams, separate compilation, ABI stability, and zero per-operation allocation. There is no performance gap on the I/O hot path.

The gap between `task<T, IoEnv>` and a coroutine-native `task<T>` is small:

| Residual cost                                   | <code>task&lt;T, IoEnv&gt;</code>              | Coroutine-native <code>task&lt;T&gt;</code> |
|-------------------------------------------------|------------------------------------------------|---------------------------------------------|
| Sender concept instantiation per task           | N (every task)                                 | 0                                           |
| Bridge instantiation                            | 0                                              | 1 (at edge)                                 |
| <code>Environment</code> template parameter     | Yes                                            | No                                          |
| Standard algorithms not designed for I/O        | Present (domain mismatch)                      | Not applicable                              |
| Task-to-task overhead per <code>co_await</code> | state_rep + virtual dispatch + scheduler check | Two pointer stores                          |
| <code>IoEnv</code> standardization dependency   | Yes                                            | No                                          |

| Residual cost                           | <code>task&lt;T, IoEnv&gt;</code> | Coroutine-native<br><code>task&lt;T&gt;</code> |
|-----------------------------------------|-----------------------------------|------------------------------------------------|
| <code>await_transform</code> contention | Yes                               | No                                             |

The costs are friction, not blockers. The gap is not insurmountable. It could probably be made to work. It is not ideal.

## 14. A Call for Reciprocity

This paper steelmanned [P3552R3](#)<sup>[1]</sup>. The author of the competing model did the work to figure out how `task<T, IoEnv>` could serve networking. The author granted every fix, traced every mechanism, and reported honestly what works and what does not.

The author asks for the same honesty from the other side.

| What the author did                                                      | What the author asks in return                                                      |
|--------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Granted every engineering fix to <code>task</code>                       | Steelman the coroutine-native model with equal rigor                                |
| Showed <code>task&lt;T, IoEnv&gt;</code> can achieve type-erased streams | Show honestly what <code>std::execution</code> loses by deferring <code>task</code> |
| Traced the mechanism end to end                                          | Trace the coroutine-native mechanism end to end                                     |
| Reported the residual costs honestly                                     | Report the residual costs of the domain partition honestly                          |

Three specific questions for the P2300 community:

- If `task` is deferred to C++29 and the domain partition is accepted, what does `std::execution` actually lose? Not in enthusiasm - in engineering. What specific use case becomes impossible?
- If bridges ([P4092R0](#)<sup>[12]</sup>, [P4093R0](#)<sup>[13]</sup>) connect the two models, what cross-domain scenario fails? Show the code.
- If coroutine-native I/O ships alongside `std::execution` in C++29, with bridges, what user is worse off than they would be with `task<T, IoEnv>` in C++26?

The committee has not seen this analysis. It has seen claims without evidence. It has seen enthusiasm over engineering. The committee deserves an honest accounting from both sides. This paper provides one side. The call is for the other.

The steelman proves collaborative intent. The author wants `std::execution` to succeed in its domain. The author wants networking to succeed in its domain. Two clean models with bridges, developed cooperatively, is the path that serves both. The committee has the opportunity to get this right. The cost of getting it right is time. The cost of getting it wrong is a decade.

---

## Acknowledgments

The author thanks Dietmar Kühl for [P3552R3](#) and [P3796R1](#) and for `beman::execution` ; Jonathan Müller for [P3801R0](#) and for documenting the symmetric transfer gap; Chris Kohlhoff for identifying the partial-success problem in [P2430R0](#); Eric Niebler, Lewis Baker, and Kirk Shoop for `std::execution` ; Steve Gerbino for co-developing the bridge implementations and [Corosio](#); Klemens Morgenstern for Boost.Cobalt and the cross-library bridges; Peter Dimov for the frame allocator propagation analysis; and Mungo Gill, Mohammad Nejati, and Michael Vandenberg for feedback.

---

## References

1. [P3552R3](#) - "Add a Coroutine Task Type" (Dietmar Kühl, Maikel Nadolski, 2025). <https://wg21.link/p3552r3>
2. [P4100R0](#) - "The Network Endeavor: Coroutine-Native I/O for C++29" (Vinnie Falco, Steve Gerbino, Michael Vandenberg, Mungo Gill, Mohammad Nejati, 2026). <https://wg21.link/p4100r0>
3. [cppalliance/corosio](#) - Coroutine-native networking library. <https://github.com/cppalliance/corosio>
4. [cppalliance/capy](#) - Coroutine I/O primitives library. <https://github.com/cppalliance/capy>
5. [D3980R0](#) - "Task's Allocator Use" (Dietmar Kühl, 2026). <https://isocpp.org/files/papers/D3980R0.html>
6. [P4088R0](#) - "The Case for Coroutines" (Vinnie Falco, 2026). <https://isocpp.org/files/papers/P4088R0.pdf>
7. [P3801R0](#) - "Concerns about the design of `std::execution::task`" (Jonathan Müller, 2025). <https://wg21.link/p3801r0>
8. [P2430R0](#) - "Partial success scenarios with P2300" (Chris Kohlhoff, 2021). <https://wg21.link/p2430r0>
9. [P2762R2](#) - "Sender/Receiver Interface For Networking" (Dietmar Kühl, 2023). <https://wg21.link/p2762r2>

10. **P4090R0** - "Sender I/O: A Constructed Comparison" (Vinnie Falco, Steve Gerbino, 2026).  
<https://isocpp.org/files/papers/P4090R0.pdf>
11. **P2470R0** - "Slides for presentation of P2300R2" (Eric Niebler, 2021). <https://wg21.link/p2470r0>
12. **P4092R0** - "Consuming Senders from Coroutine-Native Code" (Vinnie Falco, Steve Gerbino, 2026).  
<https://isocpp.org/files/papers/P4092R0.pdf>
13. **P4093R0** - "Producing Senders from Coroutine-Native Code" (Vinnie Falco, Steve Gerbino, 2026).  
<https://isocpp.org/files/papers/P4093R0.pdf>
14. **P3796R1** - "Coroutine Task Issues" (Dietmar Kühl, 2025). <https://wg21.link/p3796r1>
15. **P2762R0** - "Sender/Receiver Interface For Networking" (Dietmar Kühl, 2023). <https://wg21.link/p2762r0>
16. **P2300R10** - "std::execution" (Michał Dominiak, Lewis Baker, Lee Howes, Kirk Shoop, Michael Garland, Eric Niebler, Bryce Adelstein Lelbach, 2024). <https://wg21.link/p2300r10>
17. **P4089R0** - "On the Diversity of Coroutine Task Types" (Vinnie Falco, 2026).  
<https://isocpp.org/files/papers/P4089R0.pdf>